# *CAPÍTULO 7*

# ARREGLOS Y CADENAS

## Arreglos

Un arreglo es una lista de variables del mismo tipo. A diferencia de los lenguajes de alto nivel donde existen mecanismos para declarar y usar arreglos, en ensamblador el programador tiene que crear el código para declarar y usar los arreglos.

### Declaración de arreglos

Para declarar arreglos se utilizan las mismas directivas empleadas para declarar variables sencillas: **db** para declarar arreglos de bytes, **dw** para arreglos de palabras, **dd** para arreglos de palabras dobles y **dq** para arreglos de palabras cuádruples. La sintaxis de estas directivas, en el caso de declaraciones de arreglos, son:

[*nomArreg*] **db** *exp*[**,** *exp*]...
[*nomArreg*] **dw** *exp*[**,** *exp*]...
[*nomArreg*] **dd** *exp*[**,** *exp*]...
[*nomArreg*] **dq** *exp*[**,** *exp*]...

donde *nomArreg* es el nombre del arreglo y *exp* son expresiones constantes. El número de expresiones determina el tamaño del arreglo y sus valores se utilizan para inicializar el arreglo. Por ejemplo

```
bdtos   db   0, 1, 2, 3, 4      ; Arreglo de cinco
                                ; variables de tipo byte
                                ; inicializadas a los
                                ; valores 0, 1, 2, 3 y 4.
cresps  db   'A', 'B', 'C'      ; Arreglo de tres
                                ; variables de tipo byte
                                ; inicializadas a los
                                ; a los valores 65d, 66d
                                ; y 67d.
wdtos   dw   0123h, 2345h       ; Arreglo de cinco
        dw   4567h, 6789h       ; variables de tipo
        dw   89abh              ; palabra inicializadas
```

```
                                    ; a los valores 0123h,
                                    ; 2345h, 4567h, 6789h y
                                    ; 89abh.
```

# Declaración de arreglos usando el operador dup

En la declaración de arreglos se puede utilizar el operador **dup** para inicializar el arreglo a valores duplicados. La sintaxis del operador **dup** es la siguiente:

```
cnta dup(exp[, exp]... )
```

donde *cnta* define el número de veces que el dato definido por *exp* será repetido. El operador **dup** aparece después de una directiva para declarar variables: **db**, **dw**, etc.

Se puede anidar un operador **dup** dentro de *exp* de otro **dup**, hasta 17 niveles.

Ejemplos:

```
bdtos   db   5 dup(0)          ; Arreglo de cinco
                               ; variables de tipo byte
                               ; inicializadas a 0.

bdtosx  db   10 dup(?)         ; Arreglo de diez
                               ; variables de tipo byte
                               ; no inicializadas.

bdtos   db   5 dup(1), 0       ; Arreglo de seis
                               ; variables de tipo byte.
                               ; Las primeras cinco
                               ; inicializadas a 1 y la
                               ; última a 0.

wdtos   dw   5 dup(0, 1)       ; Arreglo de 10 variables
                               ; de tipo palabra
                               ; inicializadas a los
                               ; valores 0, 1, 0, 1,
                               ; etc.

wdtosx  dw   5 dup(4 dup(?))   ; Arreglo de 5 × 4
                               ; variables de tipo
                               ; palabra no
                               ; inicializadas.
```

# Declaración de arreglos usando la directiva de repetición rept

La directiva de repetición **rept** nos permite también declarar e inicializar arreglos como se muestra en el siguiente ejemplo:

```
label   bArreg  byte            ; Arreglo de 100 arreglos
rept    100                     ; de 21 bytes cada uno.
        db   20 dup(' '), 0     ; Cada arreglo está
endm                            ; inicializado a veinte
                                ; caracteres de espacio
                                ; seguidos de un cero
```

La primera línea de la declaración anterior utiliza la directiva **label** para declarar la etiqueta **bArreg** de tipo byte. La directiva **label** le indica al ensamblador cómo accesar a los datos que se declaran enseguida. En este caso **bArreg** será tratado como un arreglo de bytes, no reserva espacio de memoria. La sintaxis de la directiva **label** es la siguiente:

**label** *etiq tipo*

donde *etiq* es el identificador de la etiqueta y *tipo* es el tipo de los datos declarados a continuación. *tipo* puede ser: **byte**, **word**, **dword**, etc.

Como un segundo ejemplo, se tiene la siguiente declaración:

```
val = 0
label   wArreg  word            ; Arreglo de 10 variables
rept    10                      ; de tipo palabra
        dw   val                ; inicializadas a 0, 1,
        val = val + 1           ; 2, ..., 9.
endm
```

En este caso el símbolo **val** se inicializa a 0 usando la directiva =. En cada repetición el valor de **val** se ve incrementando en uno.

# Acceso a los elementos de un arreglo

El nombre de un arreglo etiqueta la dirección donde se encuentra el primer elemento de un arreglo. Esto es, el nombre del arreglo es la dirección base del arreglo. La dirección de cualquier elemento del arreglo se puede expresar como la suma de la dirección base del arreglo más la distancia de ese elemento con respecto al primer elemento del arreglo. Esa distancia puede medirse de dos formas: La distancia expresada en número de elementos, llamada también **índice**, o la distancia medida en bytes, llamada también **desplazamiento**.

El primer elemento de un arreglo tiene un índice de cero. También su desplazamiento es de cero. Si el arreglo es de bytes el índice y el desplazamiento son iguales. Para los arreglos de otros tipos el desplazamiento es un múltiplo del índice y está dado por:

desplazamiento = tamDato * índice

donde tamDato es el tamaño en bytes de cada elemento del arreglo. A diferencia de los lenguajes de alto nivel donde se accesa a los elementos de un arreglo mediante su índice, en ensamblador se accesa a los elementos de un arreglo mediante su desplazamiento. Por ejemplo sean, las siguientes declaraciones:

```
bdtosx  db      10 dup(?)
wdtosx  dw      5 dup(?)
```

entonces, las siguientes instrucciones almacenan el valor de 5 en el primer elemento, índice 0, y el valor de 6 en el tercer elemento, índice 2, de **bdtosx**.

```
        mov     [bdtosx], 5     ; bdtosx[0] = 5
        mov     [bdtosx+2], 6   ; bdtosx[2] = 6
```

y la siguientes instrucciones almacenan el valor de 500 en el primer elemento, índice 0, y el valor de 300 en el cuarto elemento, índice 3, de **wdtosx**:

```
        mov     [wdtosx], 500           ; wdtosx[0] = 500
        mov     [wdtosx+2*3], 300       ; wdtosx[3] = 300
```

En muchas ocasiones deseamos accesar a los elementos de un arreglo en una secuencia determinada por lo que sería conveniente que el índice o el desplazamiento de un elemento del arreglo estuviera en una variable o en un registro. En este caso, se puede utilizar el direccionamiento base para accesar a los elementos del arreglo. Por ejemplo, las siguientes instrucciones almacenan el valor de 6 en el elemento del arreglo **bdtosx** cuyo índice está en la variable **indice**:

```
        mov     bx, [indice]    ; bdtosx[indice] = 6
        mov     [bx+bdtosx], 6
```

Por otro lado, las siguientes instrucciones almacenan el valor de 300 en el elemento del arreglo **wdtosx** cuyo índice está en la variable **indice**:

```
        mov     bx, [indice]    ; wdtosx[indice] = 300
        sal     bx, 1
        mov     [bx+wdtosx], 300
```

Otros modos de direccionamiento que se pueden emplearse para accesar a los elementos de un arreglo son el direccionamiento indirecto, el direccionamiento indexado y el direccionamiento base-indexado.

# Modo de direccionamiento registro indirecto

En el modo de direccionamiento registro indirecto, la dirección efectiva del dato se calcula tomando como desplazamiento el contenido de uno de tres registros: **BX**, **SI** o **DI**. El segmento en los tres casos es el valor del registro de segmento de datos **DS**.

El modo de direccionamiento registro indirecto tiene la siguiente sintaxis:

**[bx]**
**[si]**
**[di]**

El direccionamiento registro indirecto se emplea normalmente para permitir que una instrucción opere en múltiples variables o elementos de un arreglo. Por ejemplo:

```
mov     ax, [bx]
inc     [word si]
dec     [byte di]
```

# Modo de direccionamiento indexado

El modo de direccionamiento indexado es similar al modo de direccionamiento **base**, sólo que utiliza los registros **SI** o **DI** en lugar de los registros **BX** o **BP**. Las referencias a **SI** o **DI** son desplazamientos con respecto al registro de segmento de datos **DS**.

El modo de direccionamiento base tiene la siguiente sintaxis:

**[si+***n***]**
**[si-***n***]**
**[di+***n***]**
**[di-***n***]**

En los dos primeros casos el desplazamiento del dato con respecto a **DS** está dado por el valor de **SI** más o menos *n* bytes. En los dos últimos casos el desplazamiento del dato con respecto a **DS** está dado por el valor de **DI** más o menos *n* bytes.

El direccionamiento indexado se emplea normalmente para accesar a los elementos de un arreglo. Por ejemplo, las siguientes instrucciones almacenan el valor de 6 en el elemento del arreglo **bdtosx** cuyo índice está en la variable **indice**:

```
mov     si, [indice]    ; bdtosx[indice] = 6
mov     [si+bdtosx], 6
```

Por otro lado, las siguientes instrucciones almacenan el valor de 300 en el elemento del arreglo **wdtosx** cuyo índice está en la variable **indice**:

```
mov     di, [indice]    ; wdtosx[indice] = 300
sal     di, 1
mov     [di+wdtosx], 300
```

# Modo de direccionamiento base-indexado

En el modo de direccionamiento base-indexado, el cálculo de la dirección efectiva del dato emplea dos registros: un registro base: **BX** o **BP** y un registro índice: **SI** o **DI**. Las referencias a **BX** son desplazamientos con respecto al registro de segmento de datos **DS** mientras que las referencias a **BP** son desplazamientos con respecto al registro de segmento de pila **SS**.

El modo de direccionamiento base-indexado tiene la siguiente sintaxis:

**[bx+si**[+*n*]**]**
**[bx+di**[+*n*]**]**
**[bp+si**[+*n*]**]**
**[bp+di**[+*n*]**]**

En los dos primeros casos el desplazamiento del dato con respecto a **DS** está dado por el valor de **BX** más **SI** o **DI** más opcionalmente *n* bytes. En los dos últimos casos el desplazamiento del dato con respecto a **SS** está dado por el valor de **BP** más **SI** o **DI** más opcionalmente *n* bytes.

El direccionamiento base-indexado se emplea normalmente para accesar a los elementos de un arreglo. Los registros **BX** o **BP** pueden contener la dirección base de un arreglo, los registros **SI** o **DI** el desplazamiento del elemento con respecto a la dirección base del arreglo y luego agregarle un posible valor n para localizar un campo en este elemento específico. Por ejemplo, las siguientes instrucciones almacenan el valor de 6 en el elemento del arreglo **bdtosx** cuyo índice está en la variable **indice**:

```
mov     bx, offset bdtosx       ; bdtosx[indice]
mov     si, [indice]            ;    = 6
mov     [bx+si], 6
```

Por otro lado, las siguientes instrucciones almacenan el valor de 300 en el elemento del arreglo **wdtosx** cuyo índice está en la variable **indice**:

```
mov     bx, offset wdtosx       ; wdtosx[indice]
mov     si, [indice]            ;    = 300
sal     si, 1
mov     [bx+si], 300
```

## Ejemplos sobre arreglos

1. El siguiente programa encuentra el mayor de un conjunto de datos de tipo palabra almacenados en un arreglo. En este programa se utiliza el direccionamiento indexado para accesar a los elementos del arreglo.

```
;*********************************************************
; MAYORNW1.ASM
;
; Este programa encuentra el mayor de un conjunto de datos
; de tipo palabra almacenados en un arreglo. El
; pseudocódigo de este programa es:
;
;    mayor = datos[0]
;    i = 0;
;
;    while(i < nDatos-1)
;    {
;       i++
;       if(mayor >= datos[i]) continue
;       mayor = datos[i]
;   }
;*********************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** DECLARACIÓN DE CONSTANTES SIMBÓLICAS **************

TAMMAX  equ     100

;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
mayor   dw      ?
nDatos  dW      ?
datos   dw      TAMMAX dup(?)

;****** CÓDIGO DEL PROGRAMA *******************************

        codeseg
inicio:
        mov     ax, @data       ; Inicializa el
        mov     ds, ax          ; segmento de datos

        mov     ax, [datos]     ; mayor = datos[0]
        mov     bx, [nDatos]    ; BX = 2 *(nDatos-1)
        dec     bx
        sal     bx, 1
        xor     si, si          ; SI = 0

while:  cmp     si, bx          ; while(SI < 2*nDatos)
        jae     endwhi          ; {
        inc     si              ;    SI++
```

```
        inc     si              ;    SI++
        cmp     ax, [si+datos]  ;    if(mayor >= datos[SI])
        jae     while           ;       continue
        mov     ax, [si+datos]  ;    mayor = datos[SI]

        jmp     while           ; }
endwhi:
        mov     [mayor], ax

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN ****************************

        end     inicio
```

2. El siguiente programa es una modificación del ejemplo anterior que utiliza el direccionamiento registro indirecto en lugar del direccionamiento indexado para accesar a los elementos del arreglo.

```
;*********************************************************
; MAYORNW2.ASM
;
; Este programa encuentra el mayor de un conjunto de datos
; de tipo palabra almacenados en un arreglo. El
; pseudocódigo de este programa es:
;
;   pDatos = datos
;   p = datos + nDatos
;   mayor = *pDatos
;
;   while(pDatos < p-1)
;   {
;      pDatos++
;      if(mayor >= *pDatos continue
;      mayor = *pDatos
;   }
;*********************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** DECLARACIÓN DE CONSTANTES SIMBÓLICAS **************

TAMMAX  equ     100
```

```
;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
mayor   dw      ?
nDatos  dW      ?
datos   dw      TAMMAX dup(?)

;****** CÓDIGO DEL PROGRAMA *******************************

        codeseg
inicio:
        mov     ax, @data          ; Inicializa el
        mov     ds, ax             ; segmento de datos

        mov     bx, offset datos   ; pDatos = datos
        mov     cx, [nDatos]       ; p = datos + (2*nDatos
        dec     cx                 ;    -1)
        sal     cx, 1
        add     cx, bx
        mov     ax, [bx]           ; mayor = *pDatos

while:  cmp     bx, cx             ; while(pDatos < p)
        jae     endwhi             ; {
        inc     bx                 ;    pDatos++
        inc     bx                 ;    pDatos++

        cmp     ax, [bx]           ;    if(mayor >= *pDatos)
        jae     while              ;       continue
        mov     ax, [bx]           ;    mayor = *pDatos
        jmp     while              ; }
endwhi:
        mov     [mayor], ax

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** CÓDIGO DE TERMINACIÓN *****************************

        end     inicio
```

## Ejercicios sobre arreglos

1. Cree un programa que sume un conjunto de datos de tipo palabra almacenados en un arreglo. Utilice el direccionamiento indexado para accesar a los elementos del arreglo.

2. Cree un programa que sume un conjunto de datos de tipo palabra almacenados en un arreglo. Utilice el direccionamiento registro indirecto para accesar a los elementos del arreglo.

# Procedimientos y arreglos

En muchas ocasiones deseamos que un procedimiento opere sobre los elementos de un arreglo. La técnica más eficiente es la de pasarle al procedimiento la dirección del arreglo y así permitirle que accese a los elementos del arreglo.

## Ejemplos sobre procedimientos y arreglos

1. El siguiente programa encuentra la primera ocurrencia de un dato dentro de un arreglo de datos de tipo palabra. El programa almacena en la variable **pos** la posición de la primera ocurrencia del valor dado por **llave**, -1 (0FFFFh) en caso contrario. En este programa se utiliza el direccionamiento base-indexado para accesar a los elementos del arreglo. Se utiliza el algoritmo de búsqueda lineal.

```
;*********************************************************
; BLINEAL1.ASM
;
; Este programa busca la primera ocurrencia del dato dado
; por llave dentro del arreglo de palabras dado por datos.
; El programa almacena en la variable pos la posición de
; llave dentro del arreglo, -1 (0FFFFh) en caso contrario.
; El pseudocódigo de este programa es:
;
;   pos = blineal(llave, datos, ndatos)
;*********************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** DECLARACIÓN DE CONSTANTES SIMBÓLICAS **************

TAMMAX  equ     100

;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
llave   dw      ?
nDatos  dW      ?
pos     dW      ?
datos   dw      TAMMAX dup(?)
```

```
;****** CÓDIGO DEL PROGRAMA ********************************

        codeseg
inicio:
        mov     ax, @data         ; Inicializa el
        mov     ds, ax            ; segmento de datos

        mov     ax, [llave]       ; AX = llave
        mov     bx, offset datos  ; BX = datos
        mov     cx, [nDatos]      ; CX = nDatos

        call    blineal

        mov     [pos], ax         ; pos = AX

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** PROCEDIMIENTOS *************************************

;*******************************************************
; BLINEAL
;
; Este procedimiento utiliza direccionamiento base indexado
; para encontrar la primera ocurrencia de llave dentro del
; arreglo datos.
;
; Parámetros:
;
;        AX = llave
;        BX = datos
;        CX = nDatos
;
; Regresa:
;
;        AX = pos si hay éxito, 0FFFFh en caso contrario
;
; El pseudocódigo para este procedimiento es:
;
;   int blineal(int llave, int *pdatos, int ndatos)
;   {
;      i = 0
;
;       while(i < nDatos)
;       {
;          if(llave == datos[i]) goto @@siesta
;          i++
;       }
;
;       return -1
;
```

```
;   @@siesta:
;       return I
;   }
;*************************************************************
proc    blineal
        push    si                  ; Preserva SI

        xor     si, si              ; SI = 0
        sal     cx, 1               ; CX = 2 * nDatos

@@whi:  cmp     si, cx              ; while(SI < 2 * nDatos)
        jae     @@endwhi            ; {
        cmp     ax, [bx+si]         ;   if(llave ==
                                    ;       datos[SI])
        je      @@siesta            ;       goto @@siesta
        inc     si                  ;    SI++
        inc     si                  ;    SI++
        jmp     @@whi               ; }
@@endwhi:
        mov     ax, 0FFFFh          ; AX = 0FFFFh
        jmp     @@fin               ; goto fin

@@siesta:
        sar     si, 1               ; SI /= 2
        mov     ax, si              ; AX = CX

@@fin:  pop     si                  ; Recupera SI
        ret
endp    blineal

;****** CÓDIGO DE TERMINACIÓN ****************************

        end     inicio
```

2. El siguiente código muestra una segunda versión del procedimiento **blineal** del ejemplo anterior que utiliza el direccionamiento registro indirecto en lugar del direccionamiento base-indexado para accesar a los elementos del arreglo.

```
;*************************************************************
; BLINEAL
;
; Este procedimiento utiliza el direccionamiento registro
; indirecto para encontrar la primera ocurrencia de llave
; dentro del arreglo datos.
;
; Parámetros:
;
;        AX = llave
;        BX = datos
;        CX = nDatos
```

```
;
; Regresa:
;
;        AX = pos si hay éxito, 0FFFFh en caso contrario.
;
; El pseudocódigo de este procedimiento es:
;
;   int blineal(int llave, int *pdatos, int ndatos)
;   {
;      pDatos = datos
;      p = pDatos + nDatos
;
;       while(pDatos < p)
;       {
;          if(llave == *pdatos) goto @@siesta
;          pDatos++
;       }
;
;       return –1
;
;   @@siesta:
;       return pDatos – datos
;   }
;*******************************************************
proc    blineal
        push    si                  ; Preserva SI

        mov     si, bx              ; pDatos = SI = datos
        sal     cx, 1               ; p = pDatos + 2*nDatos
        add     cx, bx

@@whi:  cmp     bx, cx              ; while(pDatos < p)
        jae     @@endwhi            ; {
        cmp     ax, [bx]            ;    if(llave == *pDatos)
        je      @@siesta            ;       goto @@siesta
        inc     bx                  ;    pDatos++
        inc     bx                  ;    pDatos++
        jmp     @@whi               ; }
@@endwhi:

        mov     ax, 0FFFFh          ; AX = 0FFFFh
        jmp     @@fin               ; goto fin

@@siesta:
        mov     ax, bx              ; AX = (pDatos-datos)/2
        sub     ax, si
        sar     ax, 1

@@fin:  pop     si                  ; Recupera SI
        ret
endp    blineal
```

3. El siguiente código muestra una tercera versión del procedimiento **blineal**. En el ejemplo anterior se utilizan tres apuntadores: **CX** para apuntar al final del arreglo, **SI** para apuntar al inicio del arreglo y **BX** para recorrer el arreglo. En esta versión sólo se utilizan dos apuntadores: **CX** que apunta al inicio del arreglo y **BX** que inicialmente apunta al final del arreglo y que se utiliza para recorrer el arreglo de atrás hacia adelante.

```
;*********************************************************
; BLINEAL
;
; Este procedimiento utiliza el direccionamiento registro
; indirecto para encontrar la primera ocurrencia de llave
; dentro del arreglo datos.
;
; Parámetros:
;
;        AX = llave
;        BX = datos
;        CX = nDatos
;
; Regresa:
;
;        AX = pos si hay éxito, 0FFFFh en caso contrario
;
; El pseudocódigo de este procedimiento es:
;
;   int blineal(int llave, int *pdatos, int ndatos)
;   {
;      p = pDatos + nDatos
;
;       while(p > datos)
;       {
;          p--
;          if(llave == *p) goto @@siesta
;       }
;
;       return –1
;   @@siesta:
;       return p – datos
;   }
;*********************************************************
proc    blineal
        sal     cx, 1               ; p = BX = datos
        add     cx, bx              ;  + 2*nDatos
        xchg    cx, bx              ; CX = datos

@@whi:  cmp     bx, cx              ; while(p > datos)
        jbe     @@endwhi            ; {
        dec     bx                  ;    p--
        dec     bx                  ;    p--
        cmp     ax, [bx]            ;    if(llave == *p)
        je      @@siesta            ;       goto @@siesta
        jmp     @@whi               ; }
```

```
@@endwhi:

        mov     ax, 0FFFFh          ; AX = 0FFFFh
        ret

@@siesta:
        mov     ax, bx              ; AX = (p - datos)/2
        sub     ax, cx
        sar     ax, 1
        ret
endp    blineal
```

### Ejercicios sobre procedimientos y arreglos

1. Escribe un procedimiento llamado **ppunto** que encuentre el producto punto entre dos vectores que se encuentran almacenados en dos arreglos unidimensionales. El procedimiento recibe como parámetros las direcciones de los dos arreglos en los registros **AX** y **DX** y el número de elementos en cada vector en el registro **CX**. El procedimiento regresa el producto punto en los registros **DX:AX**.

2. Escribe un procedimiento que ordene un arreglo en forma ascendente usando el algoritmo de selección. El procedimiento recibe como parámetro la direcciones del arreglo en el registro **AX** y el número de elementos en el arreglo en el registro **CX**.

# Operaciones De Cadenas

El ensamblador del 8086 posee un conjunto de instrucciones especiales para procesar arreglos de bytes y de palabras. Estas instrucciones aunque reciben el nombre de **instrucciones de cadenas** operan sobre arreglos de bytes y palabras sin importar el contenido de los arreglos. Las instrucciones de cadenas se dividen en tres grupos: Instrucciones de transferencia de cadenas, instrucciones de inspección de cadenas y prefijos de repetición de instrucciones.

Todas las instrucciones de cadenas utilizan los registros **DS:SI** y **ES:DI** para realizar su trabajo. Ambas combinaciones **DS:SI** y **ES:DI** se utilizan como índices a los arreglos sobre los que se está operando. Al igual que como lo hemos hecho con el registro de segmento de datos **DS**, debemos inicializar el registro de segmento extra **ES** para que apunte al segmento que contiene el arreglo al que va hacerse referencia mediante **ES:DI**. Si el programa contiene un sólo segmento de datos o si las cadenas sobre las que se van a operar están todas en el mismo segmento de datos la inicialización puede hacerse de la siguiente manera:

```
mov ax, @data
mov ds, ax
mov es, ax
```

Si por el contrario, queremos que el registro de segmento extra **ES** apunte a otro segmento donde está definida la variable **edato**, podemos hacer lo siguiente:

```
mov  ax, seg edato
mov  es, ax
```

Las instrucciones de cadena además de realizar su trabajo, incrementan o decrementan en forma automáticamente los registros índice que usan. Las operaciones de byte incrementan o decrementan los registros **SI**, **DI**, o ambos en uno, mientras que las operaciones de palabras incrementan o decrementan los registros **SI**, **DI**, o ambos en dos. El que las instrucciones de cadenas incrementen o decrementen los registros de índice está controlado por la **bandera de dirección D**. Si **D** = 0, entonces los registros índice se incrementan y si **D** = 1, entonces los registros índice se decrementan. Para establecer el valor de la bandera de dirección se utilizan las instrucciones: **cld** y **std**.

### cld

Coloca un cero en la bandera de dirección.

Sintaxis:

**cld**

Utilice **cld** siempre que se va a ejecutar una instrucción de cadena donde se desee que los registros **SI**, **DI**, o ambos se incrementen automáticamente.

La instrucción **cld** no afecta el resto de las banderas.

### std

Coloca un uno en la bandera de dirección.

Sintaxis:

**std**

Utilice **std** siempre que se va a ejecutar una instrucción de cadena donde se desee que los registros **SI**, **DI**, o ambos se decrementen automáticamente.

La instrucción **cld** no afecta el resto de las banderas.

## Instrucciones de transferencia de cadenas

Estas instrucciones permiten mover bytes y palabras de memoria a un registro, de un registro a memoria o directamente de memoria a memoria.

## **lods** *origen*

Carga en el acumulador **AX** o **AL** el valor del elemento de un arreglo cuyo desplazamiento con respecto del principio del arreglo está dado por **SI**.

Sintaxis:

**lods [byte** [**es:**]**si]**
**lods [word** [**es:**]**si]**

El operando de **lods** es siempre el registro **SI** que contiene el desplazamiento del dato con respecto al segmento dado por **DS**. Si el arreglo esta en el segmento apuntado por el registro de segmento extra se puede utilizar el operador **:** (dos puntos) que modifica el registro de segmento empleado por omisión. Para que **SI** contenga el desplazamiento con respecto al segmento extra en lugar del segmento de datos escribiremos **es:si** en lugar de **si**.

Cada vez que la instrucción **lods** se ejecuta, el registro **SI** se incrementa o decrementa en uno o dos para que apunte al siguiente o al anterior elemento del arreglo dependiendo de sí el valor de la bandera de dirección es cero o uno.

La instrucción **lods** no afecta las banderas.

## **lodsb | lodsw**

**lodsb** es una abreviatura de **lods [byte si]** y **lodsw** es una abreviatura de **lods [word si].**

Sintaxis:

**lodsb**
**lodsw**

## **movs** *destino***,** *origen*

Copia directamente el valor de un elemento de un arreglo cuyo desplazamiento con respecto del principio del arreglo está dado por **SI** a otro elemento de un segundo arreglo cuyo desplazamiento con respecto del principio de este segundo arreglo está dado por **DI**.

Sintaxis:

**movs [byte di], [**[**es:**]**si]**
**movs [word di], [**[**es:**]**si]**

El primer operando de **movs** es siempre el registro **DI** que contiene el desplazamiento del destino con respecto al segmento dado por **ES**. El segundo operando de **movs** es siempre el registro **SI** que contiene el desplazamiento del origen con respecto al segmento dado por **DS**. Si el arreglo origen esta en el segmento apuntado por el registro de segmento extra se puede utilizar el operador **:** (dos puntos) que modifica el registro de segmento empleado por omisión. Para que **SI** contenga el desplazamiento con respecto al segmento extra en lugar del segmento de datos escribiremos **es:si** en lugar de **si**.

Cada vez que la instrucción **movs** se ejecuta, ambos los registros **SI** y **DI** se incrementan o decrementan en uno o dos dependiendo del número de bytes copiados. Si la bandera de dirección vale 0, los registros se incrementan y si la bandera de dirección vale 1 los registros se decrementan.

La instrucción **movs** no afecta las banderas.

## movsb | movsw

**movsb** es una abreviatura de **movs [byte di], [si]** y **movsw** es una abreviatura de **movsw [word di], [si].**

Sintaxis:

**movsb**
**movsw**

## stos *destino*

Carga el valor en el acumulador **AX** o **AL** al elemento de un arreglo cuyo desplazamiento con respecto del principio del arreglo está dado por **DI**.

Sintaxis:

**stos [byte di]**
**stos [word di]**

El operando de **stos** es siempre el registro **DI** que contiene el desplazamiento del destino con respecto al segmento dado por **ES**.

Cada vez que la instrucción **stos** se ejecuta, el registro **DI** se incrementa o decrementa en uno o dos para que apunte al siguiente o al anterior elemento del arreglo dependiendo de si el valor de la bandera de dirección es cero o uno.

La instrucción **stos** no afecta las banderas.

### stosb | stosw

**stosb** es una abreviatura de **stos [byte di]** y **stosw** es una abreviatura de **stos [word di].**

Sintaxis:

**stosb**
**stosw**

## Instrucciones de inspección de cadenas

Estas instrucciones permiten comparar y revisar bytes y palabras buscando valores específicos.

### cmps *origen*, *destino*

Compara el valor del elemento de un arreglo cuyo desplazamiento con respecto del principio del arreglo está dado por **SI** con el valor del elemento de un segundo arreglo cuyo desplazamiento con respecto del principio de este segundo arreglo está dado por **DI**.

Sintaxis:

**cmps [byte** [**es:**]**si], [di]**
**cmps [word** [**es:**]**si], [di]**

El primer operando de **cmps** es siempre el registro **SI** que contiene el desplazamiento de *origen* con respecto al segmento dado por **DS**. El segundo operando de **cmps** es siempre el registro **DI** que contiene el desplazamiento de *destino* con respecto al segmento dado por **DS**. Si el arreglo *origen* está en el segmento apuntado por el registro de segmento extra se puede utilizar el operador **:** (dos puntos) que modifica el registro de segmento empleado por omisión. Para que **SI** contenga el desplazamiento con respecto al segmento extra en lugar del segmento de datos escribiremos **es:si** en lugar de **si**.

La instrucción **cmps** efectúa la resta [*origen*] - [*destino*], tira el resultado y almacena las bandera en la misma forma en que trabaja la instrucción **cmp**.

Cada vez que la instrucción **cmps** se ejecuta, ambos los registros **SI** y **DI** se incrementan o decrementan en uno o dos dependiendo del número de bytes copiados. Si la bandera de dirección vale 0, los registros se incrementan y si la bandera de dirección vale 1 los registros se decrementan.

La instrucción **cmps** afecta las banderas de sobreflujo **O**, signo **S**, cero **Z**, acarreo auxiliar **A**, paridad **P** y acarreo **C**.

## cmpsb | cmpsw

**cmpsb** es una abreviatura de **cmps [byte si], [di]** y **cmpsw** es una abreviatura de **cmpsw [word si], [di].**

Sintaxis:

**cmpsb**
**cmpsw**

## scas *destino*

Compara el valor en el acumulador **AX** o **AL** con el elemento de un arreglo cuyo desplazamiento con respecto del principio del arreglo está dado por **DI**.

Sintaxis:

**scas [byte di]**
**scas [word di]**

El operando de **scas** es siempre el registro **DI** que contiene el desplazamiento de *destino* con respecto al segmento dado por **ES**.

La instrucción **scas** efectúa la resta **AX**|**AL** - [*destino*], tira el resultado y almacena las bandera en la misma forma en que trabaja la instrucción **cmp**.

Cada vez que la instrucción **scas** se ejecuta, el registro **DI** se incrementa o decrementa en uno o dos para que apunte al siguiente o al anterior elemento del arreglo dependiendo de si el valor de la bandera de dirección es cero o uno.

La instrucción **scas** afecta las banderas de sobreflujo **O**, signo **S**, cero **Z**, acarreo auxiliar **A**, paridad **P** y acarreo **C**.

## scasb | scasw

**scasb** es una abreviatura de **scas [byte di]** y **scasw** es una abreviatura de **scas [word di].**

Sintaxis:

**scasb**
**scasw**

# Prefijos de repetición de instrucciones

Los prefijos de repetición de instrucciones son mnemónicos que preceden a las instrucciones de cadenas para crear comandos que se repitan un número de veces o hasta que se cumpla una condición.

## rep

Este prefijo repite la instrucción de cadena que le sigue un número de veces especificado por el registro **CX**.

Sintaxis:

**rep movs [byte di], [[es:]si]**
**rep movs [word di], [[es:]si]**
**rep movsb**
**rep movsw**
**rep stos [byte di]**
**rep stos [word di]**
**rep stosb**
**rep stosw**

## repe | repz

Los dos mnemónicos representan el mismo prefijo de repetición de instrucciones. Este prefijo repite la instrucción de cadena que le sigue un número de veces especificado por el registro **CX** o hasta que después de una iteración el valor de la bandera de cero **Z** valga 1.

Sintaxis:

**repe|repz cmps [byte [es:]si], [di]**
**repe|repz cmps [word [es:]si], [di]**
**repe|repz cmpsb**
**repe|repz cmpsw**
**repe|repz scas [byte di]**
**repe|repz scas [word di]**
**repe|repz scasb**
**repe|repz scasw**

## repne | repnz

Los dos mnemónicos representan el mismo prefijo de repetición de instrucciones. Este prefijo repite la instrucción de cadena que le sigue un número de veces especificado por el registro **CX** o hasta que después de una iteración el valor de la bandera de cero **Z** valga 0.

Sintaxis:

**repe**|**repz cmps [byte** [**es:**]**si], [di]**
**repe**|**repz cmps [word** [**es:**]**si], [di]**
**repe**|**repz cmpsb**
**repe**|**repz cmpsw**
**repe**|**repz scas [byte di]**
**repe**|**repz scas [word di]**
**repe**|**repz scasb**
**repe**|**repz scasw**

## Ejemplos sobre instrucciones de cadenas

1. El siguiente programa es otra variante del programa que busca un dato dentro de un arreglo de datos de tipo palabra utilizando el algoritmo de búsqueda lineal. El procedimiento empleado para hacer la búsqueda emplea la instrucción para cadenas **scasw** y el prefijo de repetición de instrucciones **repne**.

```
;*********************************************************
; BLINEAL2.ASM
;
; Este programa busca la primera ocurrencia del dato dado
; por llave dentro del arreglo de palabras dado por datos.
; El programa almacena en la variable pos la posición de
; llave dentro del arreglo, -1 (0FFFFh) en caso contrario.
; El pseudocódigo de este programa es:
;
;   pos = blineal(llave, datos, ndatos)
;*********************************************************

;****** CÓDIGO DE INICIO **********************************

        ideal
        dosseg
        model   small
        stack   256

;****** DECLARACIÓN DE CONSTANTES SIMBÓLICAS **************

TAMMAX  equ     100

;****** VARIABLES DEL PROGRAMA ****************************

        dataseg
codsal  db      0
llave   dw      ?
nDatos  dW      ?
pos     dW      ?
datos   dw      TAMMAX dup(?)
```

```
;****** CÓDIGO DEL PROGRAMA ******************************

        codeseg
inicio:
        mov     ax, @data         ; Inicializa el
        mov     ds, ax            ; segmento de datos y
        mov     es, ax            ; el segmento extra

        mov     ax, [llave]       ; AX = llave
        mov     di, offset datos  ; DI = datos
        mov     cx, [nDatos]      ; CX = nDatos

        call    blineal

        mov     [pos], ax         ; pos = AX

salir:
        mov     ah, 04Ch
        mov     al, [codsal]
        int     21h

;****** PROCEDIMIENTOS ************************************

;*************************************************************
; BLINEAL
;
; Este procedimiento utiliza la instrucción de cadenas
; scasw y el prefijo de repetición de instrucciones repne
; para encontrar la primera ocurrencia de llave dentro del
; arreglo datos.
;
; Parámetros:
;
;        AX = llave
;        DI = datos
;        CX = nDatos
;
; Regresa:
;
;        AX = pos si hay éxito, 0FFFFh en caso contrario.
;
; El pseudocódigo de este procedimiento es:
;
;   int blineal(int llave, int *datos, int ndatos)
;   {
;      n = nDatos
;
;      while(n-- > 0 && llave != *(datos++));
;
;      if(llave == *(datos-1) goto @@siesta
;
;      return –1
```

```
;
;   @@siesta:
;       return nDatos – (n+1)
;   }
;*************************************************************
proc    blineal
        push    cx                  ; Preserva CX

        cld                         ; Autoincrementa DI
        repne   scasw               ; while(CX-- > 0 &&
                                    ;      llave != [DI++]);
        cmp     ax, [di-2]          ; if(llave==datos[DI-2])
        je      @@siesta            ;    goto @@siesta

        pop     cx                  ; Restaura CX
        mov     ax, 0FFFFh          ; AX = 0FFFFh
        ret

@@siesta:
        pop     ax                  ; AX = nDatos
        sub     ax, cx              ; AX -= CX + 1
        dec     ax
        ret
endp    blineal

;****** CÓDIGO DE TERMINACIÓN ****************************

        end     inicio
```

2. El siguiente procedimiento regresa la longitud de una cadena terminada en 0.

```
;*************************************************************
; ASTRLEN
;
; Este procedimiento regresa la longitud de una cadena
; terminada en 0.
;
; Parámetros:
;
;        SI = cadena
;
; Regresa:
;
;        CX = strlen(cadena)
;
;
; El pseudocódigo de este procedimiento es:
;
;   int astrlen(char *cadena)
;   {
;      p = cadena
```

```
;
;       while(*p++);
;
;       return p – cadena - 1
;   }
;*************************************************************
proc    astrlen
        push   ax                   ; Preserva AX, DI
        push   di

        mov     di, si              ; DI = SI
        xor     al, al              ; AL = 0

        cld                         ; Autoincrementa DI
@@whi:  scasb                       ; while([DI++]);
        jnz     @@whi

        mov     cx, di              ; CX = DI - SI – 1
        sub     cx, si
        dec     cx

        popr    di                  ; Restaura DI, AX
        pop     ax
        ret
endp    astrlen
```

3. El siguiente procedimiento convierte una cadena terminada en 0 a mayúsculas.

```
;*************************************************************
; ASTRUPR
;
; Este procedimiento convierte una cadena terminada en 0 a
; mayúsculas.
;
; Parámetros:
;
;       SI = cadena
;
; Regresa:
;
;       SI = scadena
;
; El pseudocódigo de este procedimiento es:
;
;
;   char *astrupr(char *cadena)
;   {
;      n = astrlen(cadena)
;      if(¡n) goto @@fin
;
;      p = q = cadena
```

```
;       do
;       {
;          ch = *(p++)
;          if(ch >= ‘a’ && ch <= ‘z’) toupper(ch)
;          *(q++) = ch
;       }
;       while(--n > 0)
;
;       return cadena
;    }
;*************************************************************
proc    astrupr
        push    ax                  ; Preserva AX, CX, SI, DI
        push    cx
        push    si
        push    di

        call    astrlen             ; CX = strlen(cadena)

        jcxz    @@fin               ; if(!CX) goto @@fin

        mov     di, si              ; DI = SI

        cld                         ; Autoincrementa SI, DI
@@do:                               ; do
                                    ; {
        lodsb                       ;    AL = [SI++]
        cmp     al, 'a'             ;    if(AL < 'a' ||
        jb      @@sig               ;       AL > 'z')
        cmp     al, 'z'
        ja      @@sig
        sub     al, 'a'-'A'         ;    AL = toupper(AL)

@@sig:  stosb                       ;    [DI++] = AL
        loop    @@do                ; }
                                    ; while(--CX > 0)
@@fin:  pop     di                  ; Restaura DI, SI, CX, AX
        pop     si
        pop     cx
        pop     ax
        ret
endp    astrupr
```

4. El siguiente procedimiento convierte una cadena terminada en 0 que representa un entero con signo a binario.

```
;*************************************************************
; AATOI
;
; Este procedimiento convierte una cadena terminada en 0,
; que representa un número de tipo palabra con signo a
```

```
; binario. El signo si existe debe ser el primer carácter
; de la cadena. La cadena puede terminar en d, b, h
; indicando la base del número. No se permiten espacios en
; la cadena.
;
; Parámetros:
;
;       SI = cadena con el número
;
; Regresa:
;
;       AX = número en binario
;
; El pseudocódigo del procedimiento es:
;
;     int aatoi(char *s)
;     {
;        astrup(cadena)
;        l = astrlen(s)
;        signo = obtenSigno(&s, &l)
;        base = obtenBase(s, &l)
;        n = atou(s, base, l)
;        if(signo) n *= -1
;        return n
;      }
;*************************************************************
***
proc    aatoi
        push    bx              ; Preserva BX, CX, DX, SI
        push    cx
        push    dx
        push    si

        call    astrupr         ; strup(cadena)
        call    astrlen         ; CX = strlen(cadena)
        call    obtenSigno      ; DX = [SI] == '-',
                                ; SI++, CX--
        call    obtenBase       ; BX = base, CX--
        call    atou            ; AX = atou(cadena)
        cmp     dx, 0           ; if(dx == 0)
        je      @@sig           ;    goto @@sig
        neg     ax              ; ax = -ax

@@sig:  pop     si              ; Restaura SI, DX, CX, BX
        pop     dx
        pop     cx
        pop     bx
        ret
endp    aatoi
```

```
;*************************************************************
; OBTENSIGNO
;
; Este procedimiento que solo debe llamarlo aatoi, lee el
; primer carácter de una cadena que representa un número de
; tipo palabra con signo y determina si este carácter
; representa el signo del número.
;
; Parámetros:
;
;       SI = cadena con el número
;       CX = Longitud de la cadena
;
; Regresa:
;
;       CX : if([si] == '+' || [si] == '-') CX--
;       DX = [si] == '-'
;       SI : if([si] == '+' || [si] == '-') SI++
;
; El pseudocódigo de este procedimiento es:
;
;   int obtenSigno(char **s, int *l)
;   {
;      signo = 0
;      if(**s == ‘+’) goto @@pos
;      if(**s == ‘-’) goto @@neg
;      goto @@fin
;
;   @@neg:
;      signo = 1
;   @@pos:
;      (*s)++
;      *l--
;
;      return signo
;   }
;*************************************************************
proc    obtenSigno
        xor     dx, dx              ; dx = 0
        cmp     [byte si], '+'      ; if([si] == '+')
        je      @@pos               ;    goto @@pos
        cmp     [byte si], '-'      ; if([si] == '-')
        je      @@neg               ;    goto @@neg
        jmp     @@fin               ; goto @@fin

@@neg:  mov     dx, 1               ; Dx = 1
@@pos:  inc     si                  ; SI++
        dec     cx                  ; CX--
@@fin:  ret
endp    obtenSigno
```

```
;*******************************************************
; OBTENBASE
;
; Este procedimiento que solo debe llamarlo aatoi, lee el
; último carácter de una cadena que representa un número de
; tipo palabra sin signo y determina la base en que está
; representada el número. Por omisión la base es 10.
;
; Parámetros:
;
;       SI = cadena con el número
;       CX = Longitud de la cadena
;
; Regresa:
;
;       BX : if([si+cx-1] == 'B') BX = 2
;            else if([si+cx-1] == 'H') BX = 16
;            else BX = 10
;       CX : if([si+cx-1] == 'B' || [si+cx-1] == 'H' ||
;               [si+cx-1] == 'D') CX–
;
; El pseudocódigo de este procedimiento es:
;
;   int obtenBase(char *s, int *l)
;   {
;      p = s + astrlen(p) –1
;      base = 10
;      if(*p == ‘B’) base = goto @@bin
;      if(*p == ‘H’) base = goto @@hex
;      if(*p == ‘D’) base = goto @@dec
;      goto @@fin
;
;   @@bin:
;      base = 2
;      goto @@dec
;   @@hex:
;      base = 16
;   @@dec:
;      *l–
;   @@fin:
;      return base
;   }
;*******************************************************
proc    obtenBase
        push    si              ; Preserva SI
        add     si, cx          ; SI = cadena + strlen(
        dec     si              ;      cadena) – 1

        mov     bx, 10          ; base = 10


        cmp     [byte si], 'B'  ; if([si] == 'B')
        je      @@bin           ;    goto @@bin
```

```
        cmp     [byte si], 'H'    ; if([si] == 'H')
        je      @@hex             ;    goto @@hex
        cmp     [byte si], 'D'    ; if([si] == 'D')
        je      @@dec             ;    goto @@dec
        jmp     @@fin             ; goto @@fin

@@bin:  mov     bx, 2             ; base = 2
        jmp     @@dec             ; goto @@dec
@@hex:  mov     bx,16             ; Base = 16
@@dec:  dec     cx                ; CX--
@@fin:
        pop     si                ; Restaura SI
        ret
endp    obtenBase

;*************************************************************
; ATOU
;
; Este procedimiento que solo debe llamarlo aatoi,
; convierte una cadena que representa un número de tipo
; palabra sin signo a binario.
;
; Parámetros:
;
;       SI = cadena con el número
;       BX = 2, 10, 16, base del número
;       CX = strlen(cadena)
;
; Regresa:
;
;        AX = número en binario
;
; El pseudocódigo de este procedimiento es:
;
;   int atou (char *s, int base, int l)
;   {
;      n = 0
;      if(¡l) goto @@fin
;
;      do
;      {
;         n = base*n + valC(*s)
;         s++
;      }
;      while(--l > 0)
;      return n
;   }
;*************************************************************
proc    atou
        push   dx                 ; Preserva DX, DI
        push   di

        xor     ax, ax            ; n = 0
```

```
        jcxz    @@fin               ; if(!CX) goto @@fin

        xor     di, di              ; n = 0
@@do:                               ; do
                                    ; {
        mov     ax, di              ;    AX = base*n
        mul     bx
        mov     dl, [byte si]       ;    DX = [SI]
        xor     dh, dh
        call    valC                ;    DX = val([SI])
        add     ax, dx              ;    AX = base*n + DX
        mov     di, ax              ;    n = AX
        inc     si                  ;    SI++
        loop    @@do                ; }
                                    ; while(--CX > 0)

        mov     ax, di
@@fin:  pop     di                  ; Restaura DI, DX
        pop     dx
        ret
endp    atou

;*************************************************************
; VALC
;
; Este procedimiento que solo debe llamarlo atou, convierte
; un carácter que representa un número a su valor en
; binario.
;
; Parámetros:
;
;        DX = carácter
;
; Regresa:
;
;        DX = número
;
; El pseudocódigo de este procedimiento es:
;
;   int valC(char ch)
;   {
;      if(ch > '9') goto @@hex
;      return ch - '0'
;
;   @@hex:
;      return ch – ('A'- 10)
;   }
;*************************************************************
***

proc    valC
        cmp     dx, '9'
        ja      @@hex
```

```
        sub     dx, '0'
        ret

@@hex:  sub     dx, 'A' – 10
        ret
endp    valC
```

### Ejercicios sobre instrucciones de cadenas

1. Crea un procedimiento llamado **astrcat** que concatena dos cadenas. El procedimiento recibe como parámetros las direcciones de las cadenas en los registros **SI** y **DI** y concatena la cadena apuntada por **SI** a la cadena apuntada por **DI**. Los registros **SI** y **DI** deben quedar sin modificación.

2. Crea un procedimiento llamado **astrrev** que invierta una cadena. El procedimiento recibe como parámetro la dirección de la cadena en el registro **SI**. El registro **SI** debe quedar sin modificación.

3. Crea un procedimiento llamado **aitoa** que convierta un número binario a una cadena con su representación en cualquiera de las tres bases: binario, decimal y hexadecimal. El procedimiento recibe como parámetros el número a convertir en **AX**, la base en que se deberá convertir el número en **BX** y la dirección de la cadena en donde quedará el número convertido en **SI**. El registro **SI** debe quedar sin modificación.

# Bibliografía


1. Abel, Peter. Lenguaje Ensamblador y Programación para PC IBM y Compatibles. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1996.

2. Borland Int. Turbo Assembler Reference Guide. Version 1. Borland International. Scotts Valley, CA. 1988.

3. Brey, Barry B. Los microprocesadores Intel: 8086/8088, 80186, 80286, 80386 y 80486. Arquitectura, programación e interfaces. Tercera Edición. Prentice-Hall Hispanoamericana, S. A. México. 1995.

4. Godfrey, J. Terry. Lenguaje Ensamblador para Microcomputadoras IBM para Principiantes y Avanzados. Prentice-Hall Hispanoamericana, S. A. México. 1991.

5. Hyde, Randall. The Art of Assembly Language Programming. Este libro se encuentra como una serie de documento PDF en el siguiente servidor FTP:

   ftp.cs.ucr.edu/pub/pc/ibmpcdir

6. Swan, Tom. Mastering Turbo Assembler. Hayden Books. Indiana, U.S.A. 1989.

# Problemas

1. Crea un programa que encuentre la primera ocurrencia de un dato dentro de un arreglo de datos de tipo palabra utilizando el algoritmo de búsqueda binaria.

   El programa estará formado de dos módulos:

   - El primer módulo llamado **DEMO_OBB** contiene el programa principal con las variables del programa: **datos** que contiene el arreglo de datos; **nDatos** que contiene el número de datos en el arreglo; **llave** que contiene el dato a buscar y **pos** donde queda la posición de la primera ocurrencia del valor buscado.

   - El segundo módulo llamado **ORD_BBIN** contiene dos procedimientos: El procedimiento para ordenar los elementos de un arreglo visto en el ejercicio sobre arreglos y procedimientos. El segundo procedimiento implementará el algoritmo de búsqueda binaria. El procedimiento llamado **bbinaria** recibe como parámetros el valor de **llave** en **AX**, la dirección del arreglo en **BX**, el valor de **nDatos** en **CX**. El procedimiento regresa en **AX** la posición de la primera ocurrencia de llave en el arreglo, -1 (0FFFFh) en caso contrario.

2. Crea un programa que realice las cuatro operaciones fundamentales con dos datos de tipo palabra sin signo. El programa recibe los datos y la operación en una cadena llamada **soper**, por ejemplo:

   "36h + 156d"

   El resultado deberá quedar en otra cadena llamada **sresul** expresado en las tres bases, por ejemplo:

   "210d = 11010010b = D2h"

   Todas las cadenas en este programa son cadenas terminadas en 0.

   El programa estará formado por tres módulos:

   El primer módulo llamado **CALCULA** contiene tres procedimientos:

   - El procedimiento principal del programa.

   - Un procedimiento llamado **sscan** que obtiene de la cadena **soper** los dos datos y el carácter que representa la operación. El procedimiento recibe como parámetro la dirección de la cadena **soper** en el registro **SI** y regresa el primer dato en el registro **AX**, el segundo dato en el registro **BX** y el carácter que indica la operación en el registro **CX**.

   - Un procedimiento llamado **sprint** que recibe el resultado de la operación en binario y forma la cadena con la representación del resultado de la operación en decimal, binario y

hexadecimal. El procedimiento recibe como parámetros el resultado de la operación en el registro **AX** y la dirección de la cadena **sresul** en el registro **SI**.

El segundo módulo llamado **ASCII_N**, contiene los procedimientos **aatoi** y **aitoa** vistos anteriormente.

El tercer módulo llamado **STRING** contiene los procedimientos: **astrlen**, **astrup**, **astrcat** y **astrrev** vistos anteriormente.